React Is JavaScript Aimed At The Screen

- Cover
- Component
- Code
- Props
- Rerun
- State
- Props flow
- Effects
- Hooks
- Map
- Close

You bring **functions, closures and modules**. React adds three moves.



Functions

the JS you already write

Components

functions that return UI

Fresh UI

reruns on every change

Cover
Component
Code
Props
Rerun
State
Props flow
Effects
Hooks
Map
Close

You know functions, closures and the DOM.

What does React add?

A Component Is A Function Whose Return Value Is UI

Same rows on both sides, so the eye reads the pair before it reads across.

- Cover
- Component
- Code
- Props
- Rerun
- State
- Props flow
- Effects
- Hooks
- Map
- Close

JS function WHAT YOU WRITE TODAY		Component WHAT REACT RUNS	
Takes in	**Arguments**	Takes in	**Props**
Gives back	**A value**	Gives back	**An element tree**
Reruns when	**You call it**	Reruns when	**State changes**
Lives in	**Any module**	Lives in	**One file, one job**

If it returns UI, it is a component, and it typechecks like any function.

Plain JavaScript Returns Values. Components Return UI.

Read this like any function, because that is what it is.

- Cover
- Component
- Code
- Props
- Rerun
- State
- Props flow
- Effects
- Hooks
- Map
- Close

GREET.JSX

Step 1 of 2

```
1  function Greet() {
2    return (
3      <h1>
4        Hi there
5      </h1>
6    )
7  }
```

Seven lines. One function, one element.

A component runs like any function, and returns what to paint.

Props Are The Arguments Your UI Function Takes.

One line changes, and the greeting follows the caller.

- Cover
- Component
- Code
- Props
- Rerun
- State
- Props flow
- Effects
- Hooks
- Map
- Close

GREET.JSX

Step 2 of 2

```jsx
function Greet({ name }) {
  return (
    <h1>
      Hi {name}
    </h1>
  )
}
```

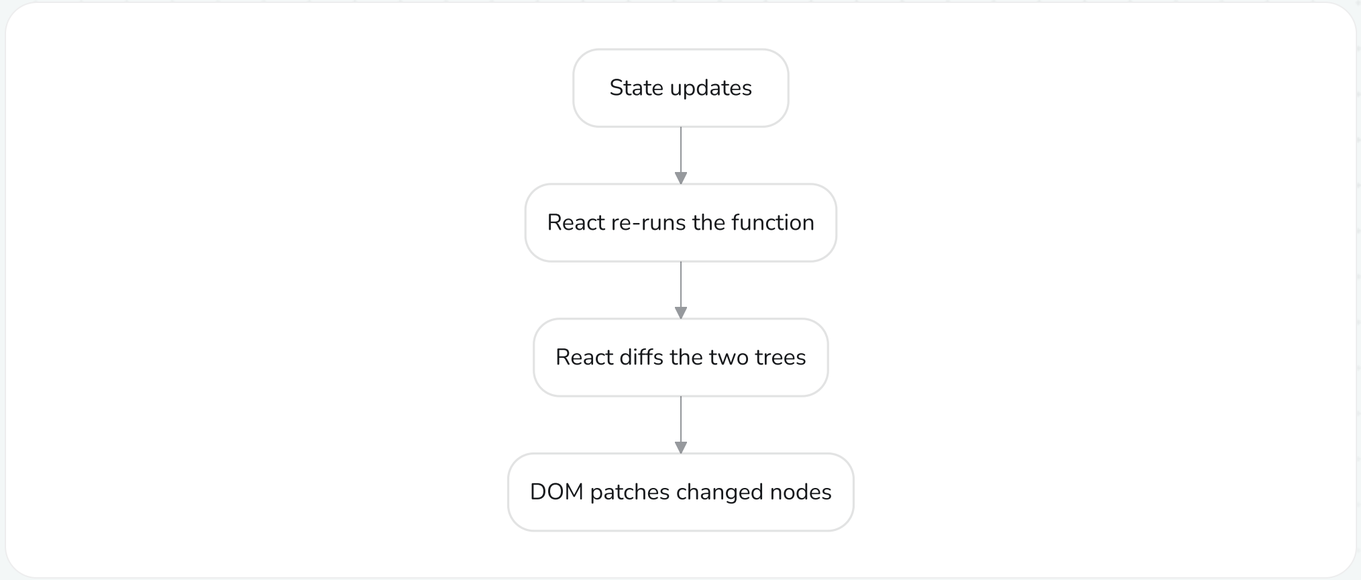
Two lines changed. The rest held still.

Props arrive like arguments, and the output follows them.

React Re-runs Your Function, Then Patches Only What Changed

You describe each frame. React moves the pixels.

- Cover
- Component
- Code
- Props
- Rerun
- State
- Props flow
- Effects
- Hooks
- Map
- Close



THE RERUN

1 call

Each state write queues one fresh run.

THE MEMORY

0 writes

Unchanged nodes cost no DOM writes.

THE BUDGET

16 ms

One frame keeps motion smooth.

You never patch the DOM. You return the next frame.

State Is Memory That Re-runs The Function

A let resets on every run. State survives it.

- Cover
- Component
- Code
- Props
- Rerun
- State
- Props flow
- Effects
- Hooks
- Map
- Close

let count PLAIN JS BINDING	
Holds	**A value**
Survives a rerun	**No, resets**
Change repaints	**No, silent**
Read it in	**Same scope**

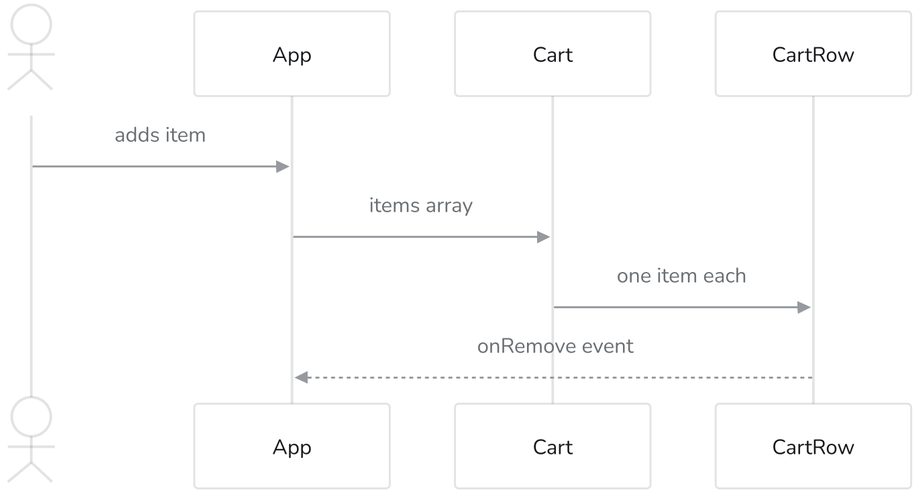
useState REACT MEMORY CELL	
Holds	**Value plus setter**
Survives a rerun	**Yes, kept by React**
Change repaints	**Yes, queues a run**
Read it in	**Same scope, stable**

A setter call is the only write that repaints the screen.

Props Flow Down, Events Call Back Up

Data falls like arguments. News rises like callbacks.

- Cover
- Component
- Code
- Props
- Rerun
- State
- Props flow
- Effects
- Hooks
- Map
- Close



props carry data down **callbacks carry news up** **state lives once, at the top**

Data falls down as props, and rises back as calls.

An Effect Syncs Your Component With The Outside World

Render first. Sync after. Clean what you set up.

- Cover
- Component
- Code
- Props
- Rerun
- State
- Props flow
- Effects
- Hooks
- Map
- Close

Render first

Paint UI from props and state. Touch nothing outside.

Exit test: the screen matches state alone

Run the sync

Fetch, subscribe or measure after paint commits.

Exit test: outside state matches inside state

Clean the last one

Return a cleanup. React runs it before the next sync.

Exit test: no timer outlives its owner

Render first, sync after, and clean what you set up.

Four Rules Keep Hooks Honest

React pairs each call by order, so order is the contract.

- Cover
- Component
- Code
- Props
- Rerun
- State
- Props flow
- Effects
- Hooks
- Map
- Close

Call hooks at the top, every render

No branch skips a call, so the order never shifts.

ALWAYS

Call hooks only from components or hooks

A plain helper cannot borrow this memory.

ALWAYS

Never call a hook inside a loop or a branch

A skipped call shifts every call below it.

NEVER

Never read state to write the same state back

Derive it during render, or store the cause.

NEVER

Same order, every render, or React pairs the wrong memory.

Five JavaScript Ideas Map To Five React Moves

The names change. Not one idea does.

- Cover
- Component
- Code
- Props
- Rerun
- State
- Props flow
- Effects
- Hooks
- Map
- Close

YOU ALREADY KNOW	REACT CALLS IT	RUNS
A function call	A component	each render
Arguments	Props	each parent render
A let binding	**useState**	**each change**
addEventListener	onClick prop	each press
fetch then innerHTML	useEffect	after paint
Rule	**State changes re-run. Props flow down.**	

Five names change, and not one idea does.

The line worth keeping after this talk

- Cover
- Component
- Code
- Props
- Rerun
- State
- Props flow
- Effects
- Hooks
- Map
- Close

React is not a language.
It is JavaScript with a rerun button.

1 function
renders the UI

1 direction
data flows down

0 DOM calls
you never touch the DOM